

Programação Orientada a Objetos

Documento complementar para o desenvolvimento do projeto
Sistema de Controle de Estacionamento de Veículos de Passeio

1. Tipos de clientes e regras de cobrança

1.1 Clientes avulsos

- São cobrados por **valor fixo por hora**, até o limite de **seis horas**.
- Caso a permanência ultrapasse seis horas, é aplicada a **tarifa de diária**.
- Se a permanência ultrapassar a **meia-noite**, uma **nova diária** é cobrada, independentemente do total de horas acumuladas no dia anterior.
- Caso o cliente avulso **se recuse a pagar**, a saída do veículo será permitida, porém o ocorrido será registrado em uma **lista de bloqueio**, impedindo futuras entradas no estacionamento.

1.2 Clientes pré-cadastrados

Clientes pré-cadastrados são aqueles que realizam previamente o cadastro de seus veículos. Eles se dividem em três categorias:

a) Professores

- Cada professor pode cadastrar **até dois veículos** vinculados ao seu **CPF**.
- A entrada no estacionamento é **gratuita**.
- Os registros são mantidos apenas para fins de **controle e seguro**, em caso de danos ao veículo.
- Apenas **um veículo por professor** pode permanecer no estacionamento simultaneamente. Caso um veículo já esteja estacionado, o segundo terá sua entrada negada.

b) Estudantes

- Cada estudante pode cadastrar **apenas uma placa de veículo**.
- O custo é um **valor fixo por ingresso**, válido desde que a entrada e a saída ocorram no **mesmo dia**.
- Caso a saída ocorra após a **meia-noite**, será cobrado um **novo ingresso**.
- O uso do estacionamento é **pré-pago**: o estudante deve manter **créditos** em sua conta.
- No momento da saída, caso o saldo seja insuficiente, o sistema permitirá que o saldo fique **negativo**, liberando a saída do veículo.
- Novas entradas serão **bloqueadas** enquanto houver saldo negativo.

c) Empresas

- As empresas podem cadastrar **quantos veículos desejarem**, vinculados ao seu **CNPJ**, correspondendo aos veículos de seus funcionários.
- Todos os veículos cadastrados podem estacionar **simultaneamente**.
- A cobrança é feita por **diária**.
- Nenhum veículo de empresa pode permanecer no estacionamento após a **meia-noite**. Caso isso ocorra, será aplicada uma **multa adicional por dia**.
- Os débitos são acumulados no cadastro da empresa e, periodicamente, é emitido um **boleto para pagamento**.
- Caso o boleto não seja quitado, a empresa passa a ser considerada **inadimplente**, e todos os seus veículos ficam **impedidos de acessar** o estacionamento até a regularização da pendência.

2. Regras gerais importantes

1. Veículos **pré-cadastrados nunca podem utilizar a modalidade avulsa**.
2. Clientes avulsos são identificados **exclusivamente pela placa do veículo**, não havendo CPF ou CNPJ associado.

3. Descontos

A EstACME pode oferecer diferentes tipos de descontos. Atualmente, o único tipo de desconto ativo é para clientes avulsos:

- **ClienteFrequente**: clientes avulsos que utilizarem o estacionamento **três vezes nos últimos cinco dias** recebem **20% de desconto**.
- O desconto é concedido **independentemente de o cliente já ter sido beneficiado anteriormente**.
- O desconto deve ser identificado internamente pela string "**ClienteFrequente**".
- O sistema deve ser projetado de forma a permitir a inclusão de **novos tipos de desconto no futuro**.

4. Funcionalidades do sistema

O sistema deve oferecer, no mínimo, as seguintes funcionalidades:

- a) **Cadastro de clientes pré-cadastrados**, armazenando CPF ou CNPJ e nome do cliente.

b) Cadastro e/ou remoção de placas de veículos junto a clientes pré-cadastrados

- Deve permitir o cadastramento ou remoção das placas dos veículos de clientes pré-cadastrados. Deve respeitar os limites de quantidade de placas por categoria de cliente.

c) Autorização de entrada de veículos:

- Verificar se o ingresso é permitido;
- Criar um registro de estacionamento contendo placa, tipo de cliente, data e hora de entrada;
- Liberar a entrada do veículo quando autorizado.

d) Processamento de saída de veículos:

- Recuperar o registro de estacionamento a partir da placa;
- Calcular o custo conforme o tipo de cliente;
- Registrar data e hora de saída;
- Registrar custo do estacionamento, identificador do desconto (ou "nenhum"), valor do desconto, valor devido e valor efetivamente pago;
- Liberar a saída do veículo.

e) Geração de relatórios gerenciais:

- Valor total arrecadado por período, ou por período e categorias de cliente (individualmente ou combinadas);
- Situação de um cliente cadastrado (veículos atualmente estacionados e saldo/débito);
- Registros de estacionamento de um cliente cadastrado em determinado período (listar todos os veículos, se houver mais de um);
- Registros de estacionamento de um cliente não cadastrado em determinado período;
- Relação dos clientes impedidos de entrar no estacionamento;
- Relação dos **10 clientes mais frequentes do ano**.

5. Especificações técnicas

- A linguagem de programação utilizada deve ser **JavaScript**.
- A implementação deve seguir rigorosamente o **diagrama de classes apresentado na Figura 1**.
- O diagrama não contempla todas as classes do sistema, porém o **conjunto mínimo especificado deve ser obrigatoriamente respeitado**.
- Recomenda-se o uso de **polimorfismo** para o cálculo do custo do estacionamento.

- É obrigatório o uso de **estruturas de dados do tipo conjunto (set) e dicionário (map/dictionary)** para:
 - Evitar o cadastro de placas duplicadas;
 - Agilizar a verificação de veículos cadastrados ou bloqueados;
 - Facilitar a identificação dos proprietários dos veículos.
- Os valores de cobrança (ingresso, valor por hora, diária, multa etc.) podem ser definidos como **constantes no código** ou carregados a partir de um **arquivo de texto**.

6. Persistência de dados

- O cadastro de clientes e os registros de estacionamento devem ser armazenados em **arquivos CSV**.
- No momento da inicialização do sistema, todos os dados devem ser **carregados integralmente para a memória**.
- As operações de atualização ocorrem exclusivamente em memória.
- Deve existir uma funcionalidade para **salvar os dados atualizados** nos respectivos arquivos CSV.
- O salvamento deve ser realizado automaticamente no **encerramento do sistema**.

7. Interface com o usuário

Os aspectos relacionados à interface com o usuário devem ser implementados em **classe separada**, que utilize as demais classes do sistema, conforme apresentado no diagrama da **Figura 1**.

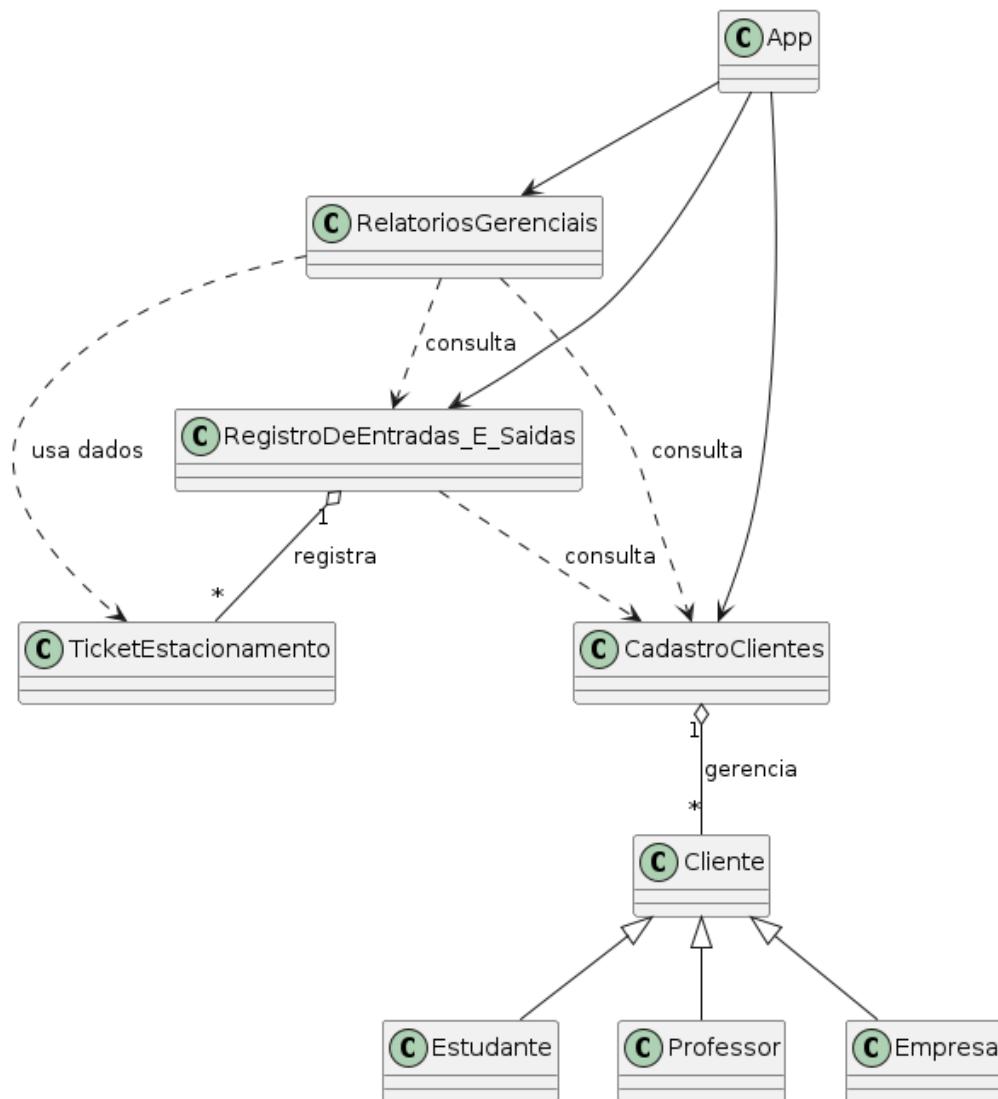


Figura 1 – Diagrama de classes

8. Formato dos arquivos CSV

A **Figura 2** apresenta um exemplo de arquivo CSV contendo o cadastro de clientes. Os dois primeiros campos correspondem ao CPF/CNPJ e ao nome do cliente. O terceiro campo pode assumir significados distintos:

- Se for **texto**, indica que o cliente é **Professor**;
- Se for **numérico**, representa o **saldo** (Estudante) ou o **débito** (Empresa).

Em seguida, aparece o campo de tipo do cliente e, por fim, a lista de placas associadas.

```
12345678901,João Silva,100,Estudante,ABC1D23
23456789012,Maria Santos,50,Estudante,DEF2E34
34567890123,Carlos Oliveira,Professor,JKL4G56,GHI3F45
45678901234,Ana Costa,Professor,MNO5H67,PQR6I78
56789012345,Tecnopuc S.A.,30,Empresa,STU7J89,VWX8K90,YZA9L01
67890123456,Empresa Inova,0,Empresa,BCD1M23,EFG2N34,HIJ3O45,KLM4P56
78901234567,Lucas Rodrigues,10,Estudante,NOP5Q67
89012345678,Beatriz Lima,Professor,QRS6R78,TUV7S89
90123456789,Tech Solutions,120,Empresa,WXY8T90
01234567890,Camila Martins,5,Empresa,ZAB9U01,CDE1V23
```

Figura 2 – Exemplo de arquivo de clientes

A **Figura 3** apresenta um exemplo de arquivo CSV contendo os registros de estacionamento. Registros incompletos representam veículos que ainda não realizaram a saída, enquanto registros completos contêm informações de entrada e saída, bem como valores cobrados.

```
ABC1D23,2025-11-27T08:30:00,2025-11-27T12:45:00,20,0,20
DEF2E34,2025-11-27T09:15:00,2025-11-27T14:20:00,25,0,25
JKL4G56,2025-11-26T07:00:00,2025-11-26T18:30:00,0,0,0
GHI3F45,2025-11-26T08:45:00,2025-11-26T17:00:00,0,0,0
STU7J89,2025-11-25T10:00:00,2025-11-26T09:00:00,70,0,70
VWX8K90,2025-11-26T14:30:00,2025-11-26T19:45:00,20,0,20
MNO5H67,2025-11-27T06:00:00,2025-11-27T11:30:00,0,0,0
BCD1M23,2025-11-26T11:20:00,2025-11-27T08:15:00,70,0,70
NOP5Q67,2025-11-27T07:45:00,2025-11-27T13:00:00,25,0,25
QRS6R78,2025-11-25T16:00:00,2025-11-25T20:30:00,0,0,0
YZA9L01,2025-11-26T09:30:00,2025-11-26T15:45:00,20,0,20
PQR6I78,2025-11-27T10:20:00,2025-11-27T16:50:00,0,0,0
EFG2N34,2025-11-25T13:00:00,2025-11-26T10:00:00,70,0,70
TUV7S89,2025-11-26T12:15:00,2025-11-26T18:00:00,0,0,0
HIJ3O45,2025-11-27T08:00:00,2025-11-27T17:30:00,20,0,20
WXY8T90,2025-11-26T15:45:00,2025-11-27T09:20:00,70,0,70
ZAB9U01,2025-11-27T11:00:00,2025-11-27T14:30:00,20,0,20
ABC1D23,2025-11-25T08:00:00,2025-11-25T11:45:00,15,0,15
KLM4P56,2025-11-26T07:30:00,2025-11-26T19:15:00,20,0,20
CDE1V23,2025-11-27T09:00:00,2025-11-27T12:30:00,20,0,20
DEF2E34,2025-11-27T15:00:00,,,
JKL4G56,2025-11-27T16:30:00,,,
STU7J89,2025-11-27T17:15:00,,,
MNO5H67,2025-11-27T14:45:00,,,
BCD1M23,2025-11-27T13:20:00,,,
NOP5Q67,2025-11-27T18:00:00,,,
YZA9L01,2025-11-27T12:10:00,,,
QRS6R78,2025-11-27T19:30:00,,,
WXY8T90,2025-11-27T11:45:00,,,
ZAB9U01,2025-11-27T20:00:00,,,
```

Figura 3 – Exemplo de arquivo com registros de estacionamento

A **Figura 4** exemplifica um método responsável pela leitura do arquivo de clientes e pelo cadastramento dos dados em memória. Neste exemplo o método faz parte da classe

“CadastroClientes” que possui um método “cadastrarCliente” que é responsável pelo cadastramento de um cliente a partir de seus dados. Este método é acionado pela rotina de leitura do arquivo para providenciar o armazenamento de cada cliente.

```
async carregarDeCSV(caminhoArquivo) {
  const fs = await import('fs');
  const conteudo = fs.readFileSync(caminhoArquivo, 'utf-8');
  const linhas = conteudo.split(/\r?\n/).filter(l => l.trim().length > 0);

  for (const linha of linhas) {
    // CSV genérico; campos por tipo:
    // Estudante: cpf,nome,creditos,tipo,placa
    // Professor: cpf,nome,tipo,placa1,placa2,...
    // Empresa: cnpj,nome,saldoDevedor,tipo,placa1,placa2,...
    const cols = linha.split(',').map(s => s.trim());

    // Detectar tipo: Estudante tem número no campo 2, outros têm texto
    const campo2 = cols[2] || '';
    const ehNumero = !isNaN(Number(campo2));

    if (ehNumero && cols.length >= 5) {
      // Estudante: cpf,nome,creditos,tipo,placa
      const [cpf, nome, creditosStr, tipo, placa] = cols;
      if (tipo.toUpperCase() === 'ESTUDANTE') {
        const creditos = Number(creditosStr);
        const cli = new Estudante(cpf, nome, '',
                                   Estudante.CUSTO_FIXO_POR_INGRESSO);
        if (!Number.isNaN(creditos) && creditos > 0)
          cli.carregarSaldo(creditos);
        if (placa)
          cli.adicionarPlaca(placa);
        this.cadastrarCliente(cli);
      } else {
        // Empresa: cnpj,nome,saldoDevedor,tipo,placa1,placa2,...
        const [cnpj, nome, saldoStr, tipoEmp, ...placas] = cols;
        const cli = new Empresa(cnpj, nome, '');
        for (const p of placas) {
          if (p.trim()) cli.adicionarPlaca(p.trim());
        }
        this.cadastrarCliente(cli);
      }
    } else if (campo2.toUpperCase() === 'PROFESSOR') {
      // Professor: cpf,nome,tipo,placa1,placa2,...
      const [cpf, nome, tipo, ...placas] = cols;
      const cli = new Professor(cpf, nome, '');
      for (const p of placas) {
        if (p.trim()) cli.adicionarPlaca(p.trim());
      }
      this.cadastrarCliente(cli);
    } else {
      // Tipo desconhecido: ignora ou lança
      console.warn('Linha ignorada (tipo desconhecido):', linha);
    }
  }
  return this.totalClientes;
}
```

Figura 4 - Método capaz de ler o arquivo de clientes